Rounds Developer Manual

Architecture, feature systems, execution flows, operations, and collaboration

---

Developer reference edition

Prepared from the current Rounds source tree and release history

20 August 2026

Contents

1 Purpose and reading guide

This manual is the technical reference for the Rounds development team. It explains what the application does, how its source is organised, which parts run on the learner's device or on the server, and how a user action passes through the system. It is intentionally written for developers and collaborators, not as learner-facing help.

Rounds is a portrait-first Expo application runtime (Expo) mobile learning application. It began as a voice-first NCLEX nursing practice product and has grown into a local-first academic platform with separate University and Uganda High School Learning portals. The application is publicly available at `https://roundsnclex-mjcssreo.manus.space`.

1.1 Core product principles

Principle	Implementation consequence
Local learning first	Practice, installed packs, saved work, topic progress, milestones, weekly rhythm, and local program choice work without an account.
Education-level separation	University and High School have distinct portals, onboarding filters, catalogues, and action guards. Nursing belongs to University.
No repeated items in a session	Question queues and topic selectors deduplicate identifiers and balance modes before a session is shown.
Original academic content	Course and topic content is subject-specific original learning material; it is not an official curriculum, grade, examination prediction, laboratory instruction, or medical advice.
Optional social identity	A Rounds account is required only for Community, notifications, private server material tools, and other protected capabilities.
Privacy by default	Device-held learning data is not sent to the server merely because the app opens. Backup export is passphrase protected and excludes session credentials.

1.2 What this manual covers

The next chapters move from app startup to individual learning systems, then server capabilities, security boundaries, test and build commands, and public collaboration. Source paths are included so a collaborator can move directly from an explanation to the relevant implementation.

2 System map

2.1 Technology stack

Layer	Technology	Responsibility
Mobile client	Expo SDK 54, React Native 0.81, React 19, Expo Router	Native screens, navigation, device APIs, local learning state, and responsive web rendering.
Styling	NativeWind 4, shared theme configuration	Portrait-first interface, sage/white light palette, dark palette, accessible tokens, and compact mobile interactions.
Client state	React context, React Query, AsyncStorage	Transient UI state, cached queries, offline progress, preferences, saved work, and learner profile.
Typed network boundary	tRPC, SuperJSON, HTTP batch link	Client calls to server procedures with an optional `x-rounds-session` header.
Server	Express, tRPC 11, Drizzle ORM, MySQL	Account sessions, Community, owner controls, PDF material metadata, notifications, and selected bounded online features.
Native services	Expo Speech, Audio, File System, Sharing, Secure Store, Haptics	Narration, recording, backup delivery, session-token storage, and touch feedback.
Quality	TypeScript, ESLint, Vitest, Expo export, esbuild	Static checks, deterministic tests, server bundle, and web release validation.

2.2 Source-tree map

Path	What belongs there
`app/`	Expo Router screens. The `(tabs)` folder contains the persistent Practice, Topics, Study, Community, and Progress tabs; top-level routes are focused learning, setup, administration, reading, and portal screens.
`components/`	Reusable native UI pieces such as safe-area containers, biometric lock, icon mapping, haptic tabs, and access gates.
`lib/`	Client-side state, device integrations, storage helpers, voice controls, adaptive selectors, encrypted backup delivery, and typed API client setup.

Path	What belongs there
`shared/`	Pure cross-platform contracts: academic catalogue, question/session evaluation, safety validators, learning units, backup crypto, and type definitions. Keep deterministic domain rules here.
`server/`	Express bootstrap, tRPC router, authentication and data-access services, storage, PDF parsing, transcription, and owner-control checks.
`tests/`	Vitest unit and integration-style domain tests. These deliberately target deterministic helpers rather than native device UI.
`docs/`	Developer-facing architecture and collaboration documents, including this manual project.
`data/`	Reviewed source content and content-processing artifacts. Do not place learner data or credentials here.

2.3 Application startup flow

When the application starts, `app/_layout.tsx` creates the provider tree and the local-first navigator. The high-level execution order is:

1. Expo loads the router entry and global NativeWind CSS.
2. `RootLayout` initialises safe-area values and the container runtime for web.
3. A single React Query client and typed tRPC client are created.
4. The app renders `AuthSessionProvider`, `LocalLearningProfileProvider`, and `BiometricAppLock` around the local-first stack.
5. The session provider checks for a locally stored opaque token. It queries `roundsAuth.me` **only** when a token exists.
6. The local profile provider restores an on-device school/program selection.
7. The stack opens the tab shell or a focused route. Learning routes do not globally redirect anonymous learners to sign-in.

```
// app/_layout.tsx — provider composition
<trpc.Provider client={trpcClient} queryClient={queryClient}>
  <QueryClientProvider client={queryClient}>
    <AuthSessionProvider>
      <LocalLearningProfileProvider>
        <BiometricAppLock>
          <LocalFirstNavigator />
        </BiometricAppLock>
      </LocalLearningProfileProvider>
    </AuthSessionProvider>
  </QueryClientProvider>
</trpc.Provider>
```

This arrangement is important: a screen can read local learner state without an account, but Community and protected server features can still read verified session state when it exists.

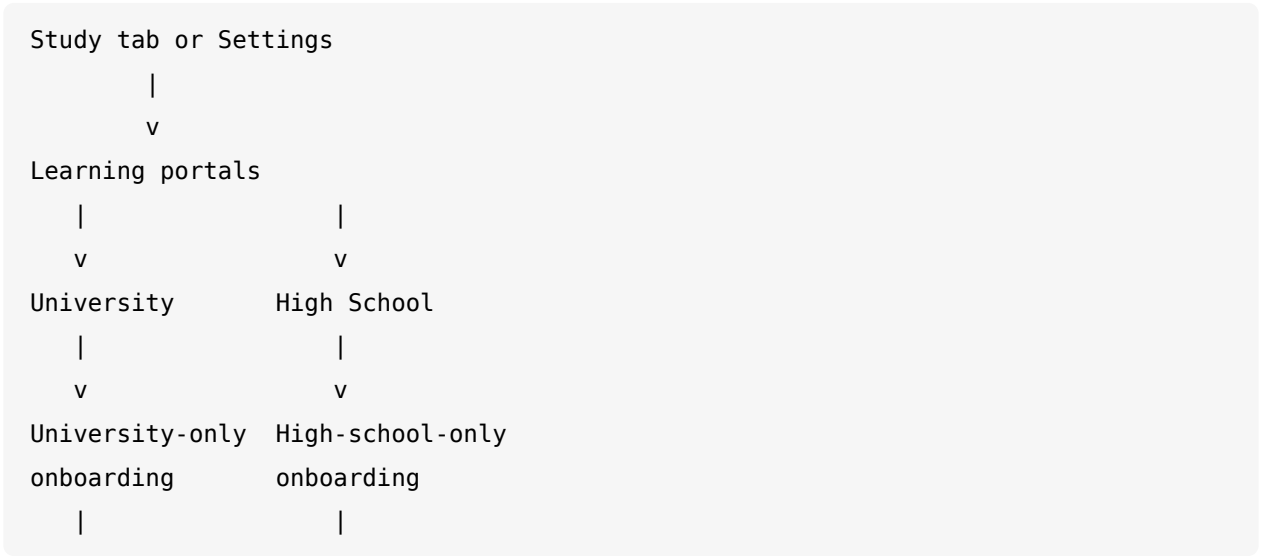
3 Navigation and learning portals

3.1 Route groups

The tab shell is the everyday home. It contains Practice, Topics, Study, Community, and Progress. The root stack contains focused experiences such as Mock Exam, Oral Exam, Voice Tutor, PDF Reader, case chains, bookmarks, owner control, notifications, and the separate education portals.
The key entry routes are:

Route	Purpose
`/learning-portals`	Level-selection gateway. Explains that University and High School are intentionally separate.
`/university-portal`	University-only entry. Contains Nursing, Foundation Year, Computing, Business, Engineering, Natural Sciences, Education, and Social Sciences.
`/high-school-portal`	Uganda High School-only entry. Contains 19 Senior 1–6 subject packs.
`/academic-onboarding?portal=...`	Portal-aware local setup. Filters choices so users cannot select a High School subject inside University setup or vice versa.
`/academic-home`	University learner home and university pack continuation.
`/high-school-home`	High School dashboard with level, scope, subject progress, milestones, weekly rhythm, and revision plan.
`/course-packs?portal=...`	Portal-specific library. It displays only one catalogue and requires a matching local path before install or launch actions.

3.2 Portal decision flow



```
   v                v
University pack  Senior 1–6 subject
library/home      library/home
```

The separation is not only visual. `shared/academic-profile.ts` derives a `LearningPortalId` from each program, and `shared/course-packs.ts` exposes `universityCoursePacks`, `highSchoolCoursePacks`, and `coursePacksForPortal`. `app/course-packs.tsx` checks a requested portal against the local profile before download or launch actions. This keeps catalogue contents, learner setup, and actions aligned.

4 Local-first data model

4.1 State ownership

Rounds distinguishes three categories of information.

Category	Examples	Storage and access
Private local learning	Program choice, session records, bookmarks, learning signals, topic progress, saved topics, revision plan, voice preference, pack installs	AsyncStorage available without account; not automatically synchronized.
Local device security	Opaque session token, biometric preference	Secure device storage and native security paths; deliberately excluded from encrypted study backup.
Protected account data	Community posts/replies/reports, alerts, account profile, private uploaded PDFs, owner actions	Server database and `protectedProcedure`; requires a valid Rounds account session.

4.2 Local learner profile

`lib/local-learning-profile.tsx` provides an application context around a validated on-device school/program selection. The provider is restored at app startup. Academic Home and High School Home prefer this profile and only fall back to the older server profile when an authenticated session exists. This is the central Local-first design change: learners may study without creating an online identity.

4.3 Local storage conventions

Local keys use a `rounds.` namespace. Significant stores include the local learning profile, high-school level and scope, topic progress, bookmarks, learning signals, course-pack install state, course round/case records, saved topics, voice preferences, and local session state. New local features should:

1. define a typed state and parser;
2. tolerate missing or malformed stored data by falling back safely;
3. keep identifiers scoped to a pack or subject;
4. write through a single helper rather than directly from many screens; and
5. add deterministic tests for parsing, state isolation, and malformed-state rejection.

5 Nursing practice systems

5.1 Practice and grading

Nursing is the reference pack and remains embedded rather than downloaded. The main Practice screen uses the typed question bank, category filters, text entry, spoken answer capture, bookmarks, and session metrics. Shared domain code evaluates answers through expected keywords and returns a correct, partial, or missed outcome with teaching feedback.

The no-repeat rule is applied at session construction: an item identifier is not added twice to the active queue. Once the queue is exhausted, a later new session can reshuffle eligible items, but an individual practice session does not repeat the same question.

5.2 Mock examination, bookmarks, and remediation

`lib/mock-exam.ts` tracks timed question navigation, flags, elapsed time, submit confirmation, and results. Bookmark state is persisted independently so a learner can return to saved questions. The post-exam review supports outcome filters—missed, partial, unanswered, flagged—and can launch a focused remediation queue without duplicating a question.

Adaptive review combines missed, partial, flagged, and saved signals in `lib/adaptive.ts` and `lib/adaptive-store.ts`. It presents the reason for prioritisation rather than making a hidden learner judgement.

5.3 Oral examination

`app/oral-exam.tsx` provides topic-first spoken practice. It prepares a selected question, speaks it with the chosen device English voice, captures or accepts a response, evaluates the answer, and can select a bounded follow-up when the response is incomplete. A source-grounded PDF feedback path is optional and only operates on the learner's own private material.

6 Course-pack architecture

6.1 Pack contract

`shared/course-packs.ts` is the catalogue authority. Each Course pack has a stable identifier, revision, title, faculty, audience, readiness, delivery mode, optional download estimate, and starter course previews. Activity kinds include recall, worked calculation, scenario, evidence reading, writing planner, logic trace, oral practice, and timed assessment.

The reusable course-round engine applies the same structural learning contract across disciplines while keeping content subject-specific. It supports completion, feedback, bookmarks, saved review, local resume, and no-repeat selection. Multi-step case chains add finite branching and local reflection. Owner controls can approve chain status but do not expose learner reflections.

6.2 University expansion

There are eight active University packs: Nursing plus Foundation Year, Computing, Business, Engineering, Natural Sciences, Education, and Social Sciences. The seven non-Nursing packs each contain 18 original local topic units across foundation, application, and reflection pathways—126 units in total. `lib/university-topic-session.ts` selects four-topic sessions with these rules:

- no duplicate unit or topic family in a session;
- saved and review-needed work receives priority when eligible;
- activity modes vary when alternatives exist;
- search is local, normalized, and limited to the active pack;
- direct-entry safety rejects units that do not belong to the active pack; and
- private completion and saved counts are displayed per pack.

6.3 Uganda High School expansion

The High School portal contains 19 active, downloadable Uganda-focused subject packs: Biology, Chemistry, Economics, Entrepreneurship, English, Physics, Mathematics, Geography, History and Civics, ICT and Digital Skills, Agriculture and Food Systems, Religious and Ethical Studies, Kiswahili, Literature, Fine Art, Technical Drawing, Food and Nutrition, Music, and Physical Education.

Each high-school subject has 18 local Topic units across three pathways, for 342 units total. A varied session has four units. The high-school selector balances new work, review-needed work, saved work, and refresh/reflection modes. It prevents duplicate unit/topic choices and keeps subject IDs isolated.

High-school-specific local systems include:

System	Behaviour
Senior 1–6 level	Learner chooses a private level. Level-matched sessions remain in the associated learning band.
Broadened scope	Optional broader sessions prioritise the learner's band and intentionally include an eligible alternate-band connection when possible.
Milestones	Private recognition at 1, 6, 12, and 18 completed topics per subject; never a grade or examination prediction.
Weekly rhythm	Tracks one or more completed topics within a UTC calendar week; reports current and longest consistency streak only on the device.
In-subject search	Normalized search over local topic title, cue, identifier, and prompt; reflection companions do not duplicate results.
Saved-topic collection	Pack-scoped, newest-first private collection with one-tap valid direct review.

7 Voice and native interaction

7.1 Spoken learning path

The shared `lib/voice.ts` layer prepares text for installed English voices, ranks choices, and protects stop operations. The `stopRoundsSpeech` helper guards a native runtime where `Speech.stop` may be missing or reject; all major voice surfaces call this helper rather than assuming a device speech engine behaves consistently.

The learner can select a device English voice in Settings. This choice applies to Practice, Oral Exam, PDF Reader, Voice Tutor, University topics, and High School topics. Spoken rationales may replay automatically after feedback if the learner enables the preference.

7.2 Microphone and transcription path

```
Learner taps record
      |
      v
Expo Audio records a supported local format
      |
      v
Audio is encoded and sent to voice.transcribe
      |
      v
Server enforces MIME type and 16 MB limit
      |
      v
Temporary signed storage URL -> transcription service
      |
      v
Transcript returns to app for review/editing
      |
      v
Shared evaluator grades the final submitted answer
```

The transcription endpoint is public but bounded. It accepts only explicitly supported audio MIME types and asks the transcription service to preserve clinical terminology. The learner can still edit the transcript or type an answer when recording/transcription is unavailable.

8 Authentication, Community, and ownership

8.1 Rounds-native accounts

`lib/auth-session.tsx` is an optional session provider. On launch it reads the stored token. If none exists, it does not query the server. If a token exists, it calls `roundsAuth.me`. Registration and sign-in send credentials to the server; the server hashes passwords, creates a short-lived Opaque session token, stores only its hash, and returns the original token for secure device storage.

```tsx
// lib/auth-session.tsx — the local-first query boundary
const me = trpc.roundsAuth.me.useQuery(undefined, {
  enabled: ready && hasStoredSession,
  retry: false,
});
```

`server/routers.ts` keeps `roundsAuth.register` and `roundsAuth.signIn` public only for account creation and authentication. It uses protected procedures for `me`, sign-out, Community, notifications, owner control, private materials, academic profile synchronization, and other account-owned resources.

8.2 Community safety loop

Community requires an account because posts, replies, reactions, reports, and alerts need an accountable identity. The shared safety validator rejects personal health information, patient details, recalled examination content, harassment, solicitation, and unsafe material. A learner can delete their own post/reply and report another item; an owner-only surface resolves reports. Reactions and replies create alerts according to stored preferences.

8.3 Owner control

Owner-only procedures require a configured Rounds owner identity. The Owner Control Center contains platform overview, learner/pack visibility, safety report resolution, and case-chain approval state. It deliberately avoids private learner reflections.

9 Server-backed learning utilities

9.1 Private PDF reader and study materials

Private learner PDFs are a protected feature. Upload validates PDF MIME type and a 4 MB content limit, stores the original privately, extracts readable sections, and writes ownership-scoped metadata. The reader supports section search, saved passages, resuming, narrated passages, and source-aware optional practice generation.

PDF practice generation and oral feedback are constrained rather than open-ended. The server tells the large language model (LLM) to treat document contents as untrusted data, use only the supplied section or file, avoid clinical advice, and return structured JSON. Private files remain owned by the uploading account and are not placed in the public source repository.

9.2 Voice Tutor and Research Updates

Voice Tutor is account-protected. It accepts a short current message plus a bounded six-turn history, applies safety redirect logic, invokes a structured model response, normalizes it, and supplies an unavailable fallback if model access fails.

Research Updates is likewise protected and online-only. It validates a nursing topic, searches only an explicit trusted-domain allowlist through a bounded model tool call, normalizes cited results, and returns a clear offline/error message instead of inventing sources.

10 Encrypted local backup export

The Settings screen offers a learner-controlled encrypted `.rounds` file export. The passphrase must be confirmed and at least eight characters; it is never stored. The flow is:

1. `collectLocalStudyData()` lists local `rounds.` keys and filters them through a strict allowlist.
2. The backup payload adds a schema version and ISO export time.
3. PBKDF2-HMAC-SHA-256 derives a key from the passphrase and fresh random salt.
4. AES-256-GCM encrypts the payload with a unique nonce and authenticated envelope metadata.
5. The app writes JSON to the native cache directory and opens the device share sheet. On web it triggers a browser download.

```
// lib/local-backup.ts — the data collection boundary
const eligibleKeys = (await storage.getAllKeys())
  .filter((key) => key.startsWith("rounds."))
  .sort();
const records = eligibleKeys.length ? await storage.multiGet(eligibleKeys) : [];
return {
  schemaVersion: BACKUP_SCHEMA_VERSION,
  exportedAt: now.toISOString(),
  storage: filterStudyStorageRecords(records),
};
```

The allowlist includes learner profile, practice/session state, bookmarks, learning signals, installed packs, course/high-school progress, drafts, and voice preferences. It excludes password data, opaque sessions, biometric settings, server-backed PDF caches, and device-security state. Tests cover allowlisting, envelope confidentiality, tamper and wrong-passphrase rejection, and recovery.

11 End-to-end execution flows

11.1 Local topic session

```
Open correct portal
    -> choose/restore local profile
    -> select installed subject/program pack
    -> load pack-scoped local progress + saved IDs
    -> adaptive selector removes duplicates and balances modes
    -> show four-unit session
    -> learner answers/saves/listens
    -> store completion and optional reflection locally
    -> recompute progress, milestone, saved counts, and next session candidates
```

No Community request, account lookup, or server database write is required for that flow.

11.2 Protected Community action

```
Learner enters Community
    -> no local session: show secure access gate
    -> register/sign in: server validates credentials and returns opaque token
    -> token stored in secure device storage
    -> tRPC adds x-rounds-session header
    -> protected server procedure resolves learner
    -> validate post/reply text and ownership
    -> persist record, optionally create notification
    -> invalidate relevant client query and refresh UI
```

11.3 Server request boundary

The tRPC client in `lib/trpc.ts` retrieves the local opaque token asynchronously and adds it as `x-rounds-session`. The Express bootstrap permits that header in credentialed CORS preflight and mounts tRPC under `/api/trpc`. New protected routes should use a shared validation function, a Zod input contract, an ownership check in the database layer, and deterministic tests for both accepted and rejected cases.

12 Testing, builds, and release process

12.1 Test strategy

Rounds uses Vitest for deterministic tests. The project contains 41 active test files plus one intentionally skipped file at the latest validated release, with 130 passing tests and one skipped test. Tests target pure modules and boundaries such as answer evaluation, no-repeat queues, adaptive priority, store parsing, portal isolation, encryption envelopes, Community permissions, notification rules, PDF safety, account/session behavior, voice text preparation, and topic selection.

For a native feature, do not depend on browser-driven tests of Expo runtime behavior. Move deterministic logic into `shared/` or a test-compatible `lib/` utility and unit-test it with controlled inputs. Native effects—recording, sharing, haptics, speech, biometrics—should be guarded by platform checks and modeled through small helper functions.

12.2 Required validation commands

```
# Static checks
pnpm check
pnpm lint

# Deterministic suite
pnpm test -- --run

# Production server bundle
pnpm build

# Browser-ready static export
npx expo export --platform web
```

Before a checkpoint, review the project task tracker and mark each completed release task. A checkpoint records source, dependencies, and project state. The current live domain is `roundsnclex-mjcssreo.manus.space`.

12.3 Developer change checklist

Step	Required action
1	Identify whether the feature is local-only, protected account data, owner-only data, or a bounded public utility.

Step	Required action
2	Add an unchecked task to the project tracker before implementation and choose a pack/portal scope where applicable.
3	Keep domain rules in shared/testable utilities; avoid duplicating logic across screens.
4	Use `ScreenContainer`, Safe Area handling, native touch feedback, and portrait-first layouts.
5	Preserve University/High School isolation and do not put Nursing into the High School portal.
6	Validate malformed persisted data, ownership, pack identity, and no-repeat boundaries.
7	Run type, lint, test, build, and web-export validation; visually verify affected mobile routes.
8	Update documentation, checkpoint the release, and publish only after the owner approves external sharing.

13 Feature evolution record

The project tracker records 336 completed development tasks at the time of this manual. The following grouped history explains the important upgrades without forcing a developer to read every intermediate checkpoint.

13.1 Foundation and NCLEX learning

The first releases established a native question-bank model, clinical categories, keyword-based grading, local practice sessions, speech delivery, microphone recording and transcription, typed fallback, manual/auto-advance practice, categories, progress, Settings, haptics, and accessibility labels. The nursing content pipeline then added extraction, normalization, answer pairing, respectful topic organization, de-duplication, and no-repeat queue behavior across a large question bank.

Mock exams, bookmarks, post-exam review, remediation, adaptive learning signals, and oral practice expanded the learning loop. Voice work later added device English-voice ranking, pacing, replay and stop controls, optional spoken rationales, failure recovery, and the guarded speech-stop repair.

13.2 Secure social and private learning

Rounds-native account registration, sign-in, hashed passwords, opaque sessions, protected procedures, Community posts/replies/reactions/reports, notifications, account export, and biometric preference were added. The local-first redesign then removed mandatory sign-in from ordinary learning and moved account creation to Community/notifications only. This made offline study a first-class workflow rather than a limited anonymous mode.

13.3 Academic platform expansion

The app introduced program-aware onboarding, a reusable course-pack registry, equal active-pack controls, shared learning rounds, calculation/logic activity players, scenarios, multi-step case chains, finite

branches, reflection reviews, and owner approval state. Every active University and High School pack uses the same architecture while retaining subject-specific content and safety language.

Uganda High School grew from initial core subjects to 19 active packs, then to Senior 1–6 level controls, revision planning, 342 original topic units, varied four-topic sessions, level scope, milestones, weekly rhythm, local search, saved-topic review, and shared voice controls. University packs later gained 126 original topic units and the same no-repeat, search, saved review, progress, reflection, and device-voice capabilities.

13.4 Privacy, backup, and portals

The encrypted backup release created a learner-controlled recovery route without a server upload. The latest portal release separated University and High School navigation from first entry through course action. It introduced a learning gateway, portal-specific onboarding, portal-specific catalogues, and action guards that demand a matching private study path.

14 Collaboration and public source sharing

The public source package is prepared with a project README, `CONTRIBUTING.md`, `SECURITY.md`, and `docs/local-configuration.md`. The repository ignores environment files, log directories, generated builds, native credentials, and editor artifacts. Before a public push, audit tracked files for credentials and do not include learner exports, session tokens, PDF uploads, database dumps, or deployment secrets.

For collaborators, use a branch and pull-request workflow:

1. branch from `main` for one coherent change;
2. use the correct portal and local/protected data boundary;
3. add or update deterministic tests;
4. run the validation commands; and
5. ask for review of learning safety, privacy, accessible mobile layout, and source quality before merge.

Choose a repository licence before granting broad public reuse. A public repository makes code viewable; collaborator write access should be granted through the source host only to named, trusted contributors.

15 Glossary

AES-256-GCM: Authenticated encryption used for the learner-controlled local study-backup file.

AsyncStorage: On-device key-value persistence used for private learner state and offline continuity.

Course pack: A subject or program container defining its audience, delivery model, active learning units, and local install state.

***Expo* – Expo application runtime**: The React Native application platform used by Rounds for Android, iOS, and web delivery.

***LLM* – large language model**: A server-invoked model used only for bounded features such as the Voice Tutor, trusted research summaries, and private PDF practice generation.

Local-first: A product boundary in which learning works from device-held data without an account or mandatory server request.

Opaque session token: A random server-issued value stored locally; the server stores a hash rather than the original value.

PBKDF2: Passphrase key-derivation function used before local backup encryption.

Learning portal: The top-level University or High School route that isolates education-level catalogues and navigation.

Protected procedure: A server procedure that requires a verified Rounds account session.

Expo Router: File-based navigation framework used for tabs and protected or focused application screens.

Topic unit: A local, subject-specific learning item selected into an adaptive no-repeat study session.

tRPC: Typed client-to-server procedure layer used by account, Community, PDF, notification, and other server-backed capabilities.